

Top sheet

(each student must have their own individual lab manual)

Index sheet

Project Name:

Electronic Cash Counter

EXPERIMENT 1

Objective 1: Problem Analysis and Project Planning

- **Problem Statement:**
The Account Division of a banking sector requires a system to streamline its transaction processes and provide a better interface for its users. Current manual or semi-automated systems are insufficient to meet customer expectations for real-time data access and transaction processing.
- **Project Scope:**
The system will automate key banking functions, including account management, fund transfers, cheque book requests, and reporting. The project will focus on improving user experience through a secure, user-friendly interface that integrates with a database for real-time data handling.
- **Infrastructure Required:**
A web-based system with a secure database backend, SSL encryption for security, and 24/7 availability. The system should handle peak loads efficiently and allow for future scalability.

EXPERIMENT 2

Objective 2: Software Requirement Specification (SRS):

1. Introduction :

Purpose

The purpose of this SRS document is to define the functional and non-functional requirements of the **Electronic Cash Counter** system. The system is designed to automate banking transactions, providing both customers and administrators with a secure, user-friendly platform for managing accounts, transferring funds, and handling various other banking operations.

Scope

This project focuses on developing a secure web-based application that facilitates real-time banking operations. It is aimed at automating key banking activities such as account creation, deposits, withdrawals, fund transfers, chequebook requests, and generating reports. The system will also include administrative functionalities, such as managing user accounts, providing customer support, and ensuring secure transactions.

Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **SSL:** Secure Socket Layer
- **PK:** Primary Key

- **FK:** Foreign Key
- **UAT:** User Acceptance Testing

Modules/Phases:

1. **Bank Account Module:**

Customers log in using their credentials to perform operations like viewing transactions, changing personal information, paying bills, and transferring money.

2. **Bank Account Administrator Module:**

Administrators can log in to manage customer accounts, open new accounts, perform deposits, withdrawals, transfers, and delete accounts.

Functional Requirements:

- Transaction detail retrieval.
- Funds transfer between accounts.
- Cheque book request.
- Monthly statement viewing.
- Balance inquiry.

Non-Functional Requirements:

- Secure access using SSL.
- 24/7 system availability.
- High performance during peak usage times.
- Scalable architecture for future extensions.

2. Overall Description

Product Perspective

The **Electronic Cash Counter** system is intended to automate common banking operations, improve customer interaction, and enhance transaction security. It is a stand-alone system with a web interface designed to support both individual users and administrators.

Product Features

- User-friendly interface for customers to manage accounts and perform transactions.
- Real-time updates on account balances, transaction history, and transfers.
- Secure data handling with multi-level authentication.
- Administrative tools for account creation, management, and reporting.

User Classes and Characteristics

- **Customers:** Individuals with access to their accounts for viewing balances, making transactions, and managing personal information.
- **Administrators:** Bank personnel who manage user accounts, process customer requests, and monitor system performance.

Operating Environment

- **Platform:** Web-based application.
- **Operating System:** Windows/Linux/macOS with a web browser.
- **Browser Compatibility:** Chrome, Firefox, Edge, and Safari.
- **Database:** SQL-based database management system (e.g., MySQL, Oracle).
- **Security:** SSL encryption for all user sessions.

Design and Implementation Constraints

- The system must support simultaneous multi-user access.
- It should comply with industry standards for security, including encryption and multi-level authentication.
- Response time for all operations must be under 3 seconds.

Assumptions and Dependencies

- The system assumes that users have access to the internet and a web browser.
- The backend database will be continuously available for real-time transaction processing.

EXPERIMENT 3

Objective 3: Data Modeling

1. Data Dictionary

Customer table

Name	Null?	Type
Customer_id (PK)	NOT NULL	INTEGER
Cust_first_name		VARCHAR2(20)
Cust_last_name		VARCHAR2(20)
DOB		VARCHAR2(20)
Gender		VARCHAR2(2)

Login table

Name	Null?	Type
Customer_id (FK)		INTEGER

Password		VARCHAR2(30)
Username		VARCHAR2(30)

Customer Detail table

Name	Null?	Type
Customer_id (FK)	NOT NULL	INTEGER
City		VARCHAR2(20)
State		VARCHAR2(20)
Zip		VARCHAR2(20)
Phone Number		NUMBER(10)
Email id		VARCHAR2(20)

Credit Card table

Name	Null?	Type
Request Number	NOT NULL	INTEGER
Name		VARCHAR2(30)
Profession		VARCHAR2(30)
Annual Income		INTEGER
Address		VARCHAR2(30)
City		VARCHAR2(30)
Telephone Number		VARCHAR2(30)
Card type		VARCHAR2(30)

Account table

Name	Null?	Type
Account Number (PK)	NOT NULL	NUMBER(8)
Customer_id (FK)	NOT NULL	INTEGER
Min_Balance		NUMBER(8)
Current_balance		NUMBER(8)
Recommended_by		VARCHAR2(20)
Nominee		VARCHAR2(20)
Type_of_account		VARCHAR2(20)
Date_of_opening		VARCHAR2(20)
Date_of_access		VARCHAR2(20)

Branch locator table

Name	Null?	Type
Location	NOT NULL	VARCHAR2(30)
Branch_city		VARCHAR2(20)
Address		VARCHAR2(30)

Employee table

Name	Null?	Type
Employee_id (PK)	NOT NULL	NUMBER(10)
Name		VARCHAR2(20)
Working_from		VARCHAR2(20)
Age		NUMBER(10)

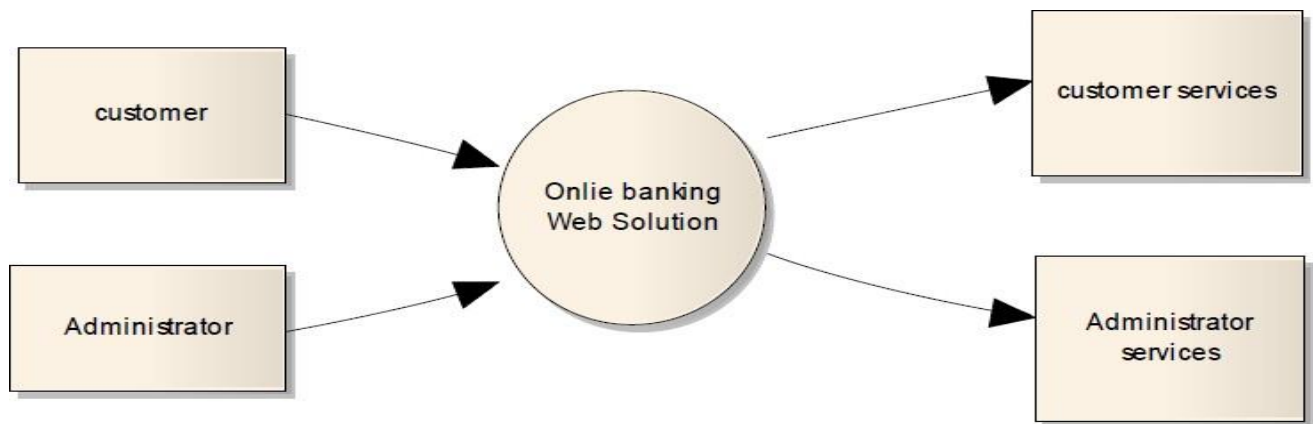
Transaction(transfer-funds) table

Name	Null?	Type
Trans_id	NOT NULL	NUMBER(10)
Acc_no		NUMBER(10)
Account_to		NUMBER(10)
Amount		NUMBER(10)
Transaction_date		VARCHAR2(20)
Trans_no		INTEGER
description		VARCHAR2(30)

Transaction type table

Name	Null?	Type
Transaction Number (PK)	NOT NULL	INTEGER
Account Number (FK)	NOT NULL	INTEGER

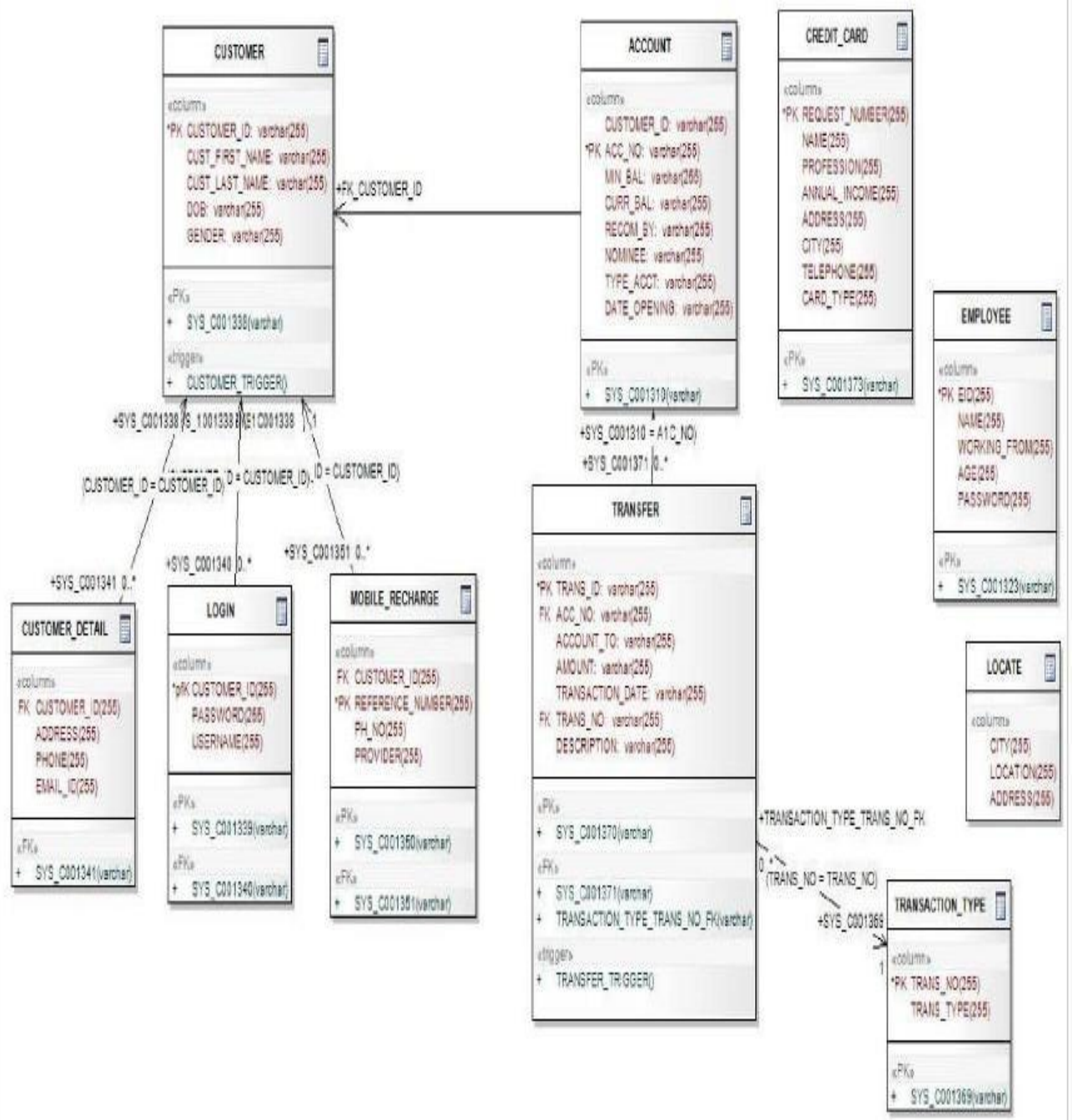
2. Context Level Diagram



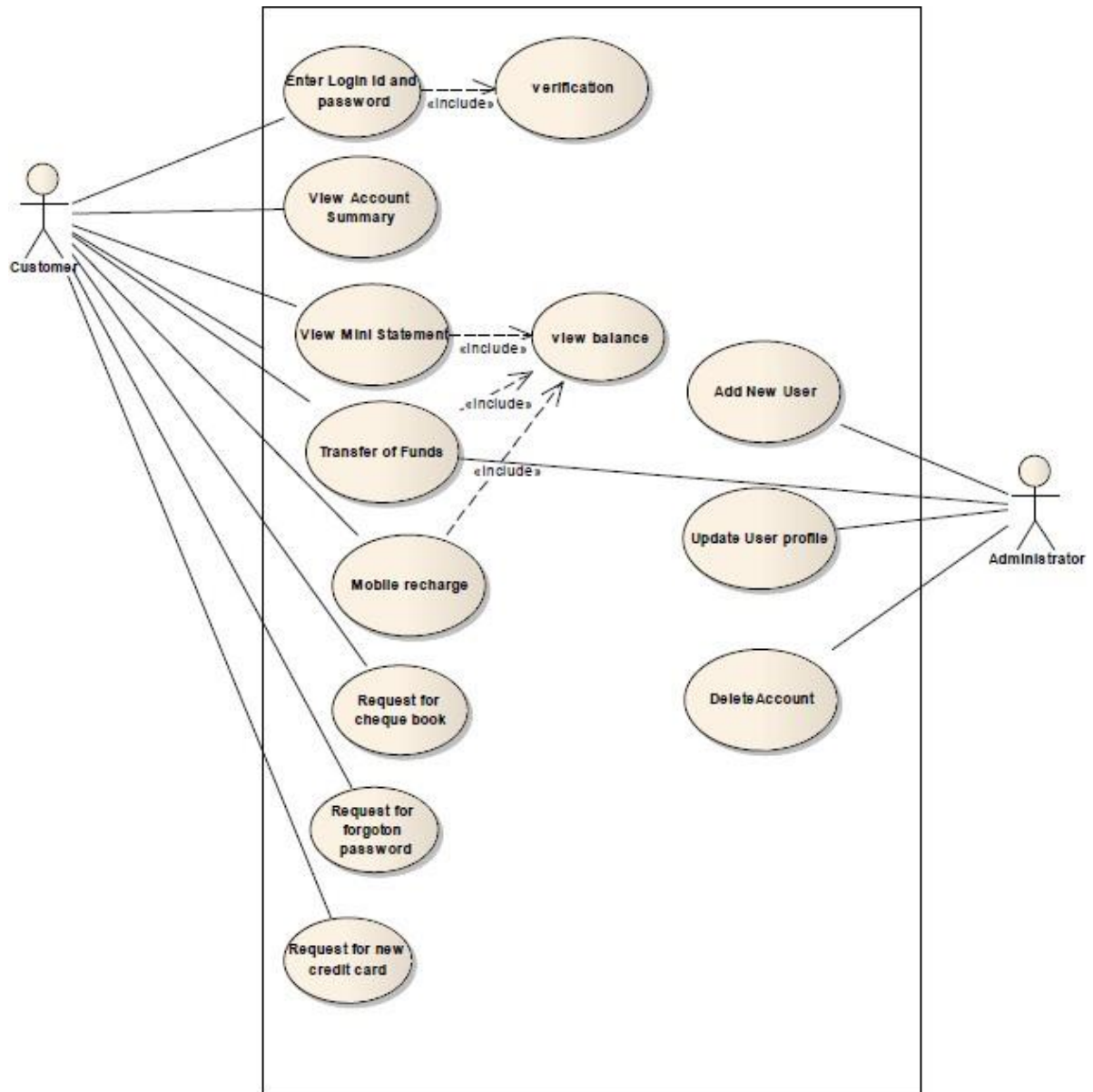
EXPERIMENT 4

Objective 4: Software Designing

1) Class diagram:

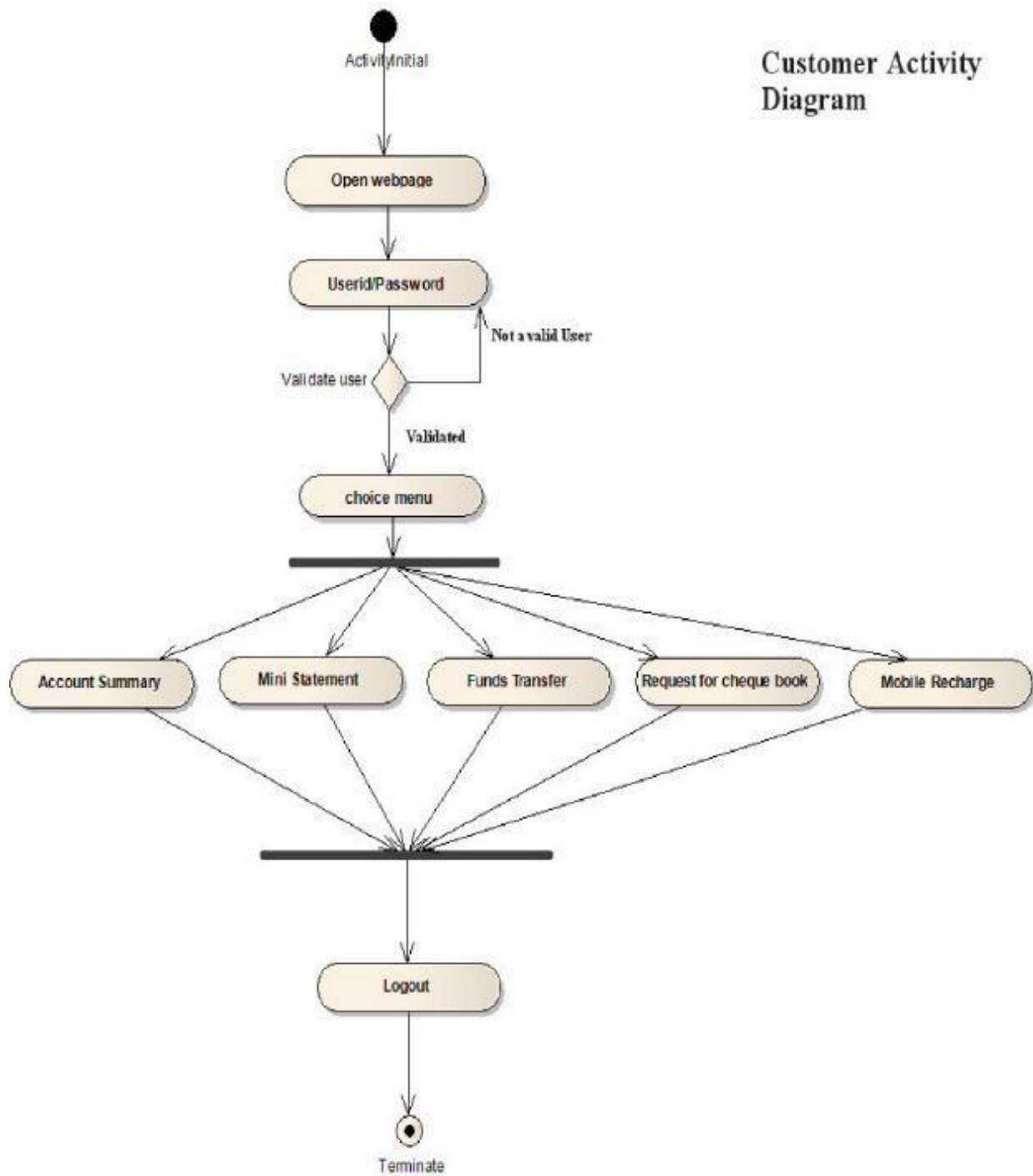


2) Use case Diagram

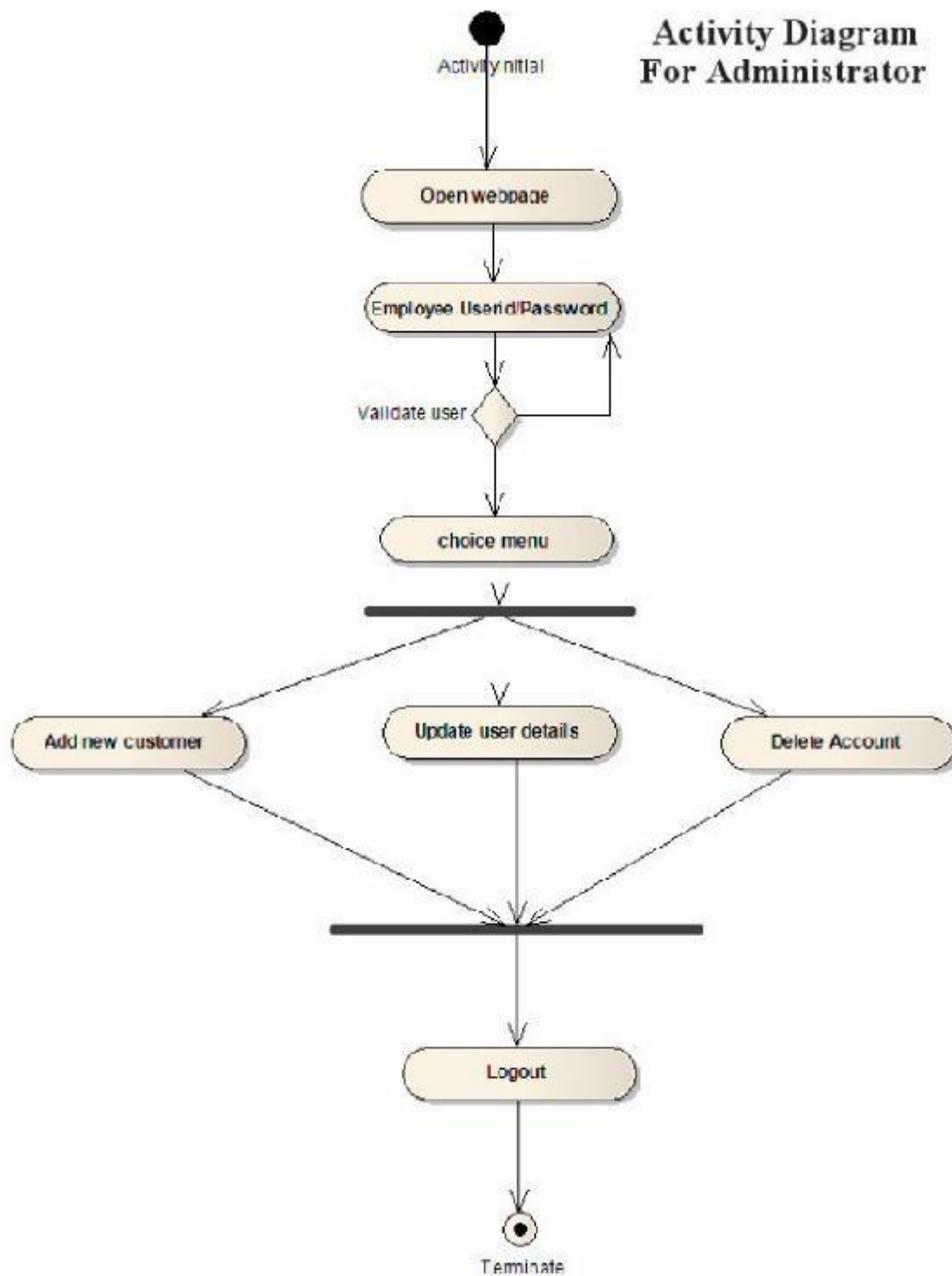


3)Activity Diagram

(3.1)Customer Activity Diagram



(3.2)Activity Diagram for Administrator



EXPERIMENT 5

Objective 5: Prototype model – Develop the product prototype.

(screenshots of the front end to be pasted here)

EXPERIMENT 6

Objective : Software Requirement Analysis – Describe the individual Phases/modules of the project and identify deliverables. Identify functional and non-functional requirements.

Software Requirement Analysis

Modules/Phases:

1. Bank Account Module

- **Description:** This module enables customers to log in and manage their bank accounts. Customers can view transaction history, modify personal information, pay bills, and transfer funds between accounts.
- **Key Features:**
 - **User Login:** Customers authenticate using their credentials.
 - **Transaction History:** Customers can retrieve details of their past transactions.
 - **Personal Information Management:** Customers can update their personal information.
 - **Bill Payment:** Customers can initiate and manage bill payments.
 - **Funds Transfer:** Customers can transfer money between their accounts.

2. Bank Account Administrator Module

- **Description:** This module allows bank administrators to manage customer accounts and perform essential banking operations.
- **Key Features:**
 - **Admin Login:** Administrators authenticate using their credentials.
 - **Account Management:** Administrators can open new customer accounts and delete existing ones.
 - **Transaction Processing:** Administrators can process deposits, withdrawals, and transfers.

Functional Requirements:

1. Transaction Detail Retrieval:

- The system shall allow customers to retrieve and view their transaction details, including date, amount, and transaction type.

2. Funds Transfer Between Accounts:

- The system shall enable customers to transfer funds between their own accounts securely.

3. Cheque Book Request:

- The system shall provide customers with the ability to request a new cheque book.

4. Monthly Statement Viewing:

- The system shall allow customers to view and download their monthly account statements.

5. Balance Inquiry:

- The system shall provide customers with the current balance of their accounts.

Non-Functional Requirements:

1. **Secure Access Using SSL:**
 - The system shall implement Secure Socket Layer (SSL) protocols to ensure secure communication between the client and server.
2. **24/7 System Availability:**
 - The system shall be available to users 24 hours a day, 7 days a week, ensuring uninterrupted service.
3. **High Performance During Peak Usage Times:**
 - The system shall be designed to handle high transaction volumes during peak usage times without degradation of performance.
4. **Scalable Architecture for Future Extensions:**
 - The system architecture shall be scalable to accommodate future enhancements and additional features.

EXPERIMENT 7

Objective : Testing phase-related documents.

1. Design Documents

- **Purpose:** Outline the architecture and design of the software solution.
- **Components:**
 - **Architecture Design Document:** Details the overall system architecture, including high-level components and their interactions.
 - **Detailed Design Document:** Provides in-depth descriptions of modules, interfaces, and data structures.
 - **User Interface Design Document:** Outlines the UI/UX aspects, including wireframes and design elements.
 - **Database Design Document:** Contains the schema design, entity-relationship diagrams, and data dictionary.
 - **Component Design Specifications:** Individual design specifications for each module, including algorithms, methods, and performance considerations.

2. Testing Phase-Related Documents

- **Purpose:** Ensure the software meets specified requirements and is free of defects.
- **Components:**
 - **Test Plan:** Describes the scope, approach, resources, and schedule for testing activities.
 - **Test Case Document:** Lists test cases to be executed, including input data, expected results, and execution conditions.
 - **Test Script:** Detailed scripts for automated testing, specifying the steps and expected outcomes.
 - **Defect Tracking Document:** Records defects identified during testing, including status, severity, and resolution steps.

- **Test Summary Report:** Provides an overview of testing activities, outcomes, and recommendations for release.

3. Integration of Design and Testing Documents

- Ensure that the design documents support the testing phase by clearly defining requirements and specifications that testing must validate.
- Use traceability matrices to link requirements in design documents to corresponding test cases.

4. Tools and Techniques

- Consider using tools such as UML for design documentation and test management software (e.g., JIRA, TestRail) for managing testing documents.
- Implement version control systems (e.g., Git) for managing changes to design documents.

EXPERIMENT 8

Objective :Preparation of Software Configuration Management and Risk Management-related documents.

1. Software Configuration Management (SCM) Documents

- **Purpose:** Ensure the integrity, consistency, and traceability of software throughout its lifecycle.
- **Components:**
 - **Configuration Management Plan:**
 - **Overview:** Outlines the processes and practices for managing software configurations.
 - **Key Sections:**
 - Configuration identification
 - Version control procedures
 - Change management processes
 - Configuration audits and reviews
 - **Configuration Item (CI) List:**
 - Defines all configuration items, including software components, documentation, and hardware.
 - Specifies version numbers and status of each item.
 - **Change Request Document:**
 - Captures requests for changes to software configurations, including justification and impact analysis.
 - **Baseline Documents:**
 - Establishes formal baselines for software releases, including functional and non-functional requirements.
 - Describes the process for establishing and approving baselines.
 - **Configuration Status Report:**
 - Provides an overview of the current status of all configuration items and any outstanding change requests.

2. Risk Management Documents

- **Purpose:** Identify, analyze, and mitigate risks throughout the software development lifecycle.
- **Components:**
 - **Risk Management Plan:**
 - **Overview:** Outlines the approach for managing risks, including methodologies and roles.
 - **Key Sections:**
 - Risk identification processes
 - Risk assessment techniques
 - Risk mitigation strategies
 - Monitoring and reporting procedures
 - **Risk Register:**
 - Maintains a log of identified risks, including descriptions, impact assessments, likelihood, and mitigation strategies.
 - Includes risk owners and status updates.
 - **Risk Assessment Matrix:**
 - Visual representation of risks, categorizing them based on likelihood and impact.
 - Helps prioritize risks for further action.
 - **Risk Mitigation Plan:**
 - Details specific strategies for addressing high-priority risks.
 - Includes contingency plans and resources required for implementation.
 - **Risk Monitoring Report:**
 - Provides updates on risk statuses, including newly identified risks and changes to existing risks.
 - Summarizes the effectiveness of mitigation strategies.

3. Integration of SCM and Risk Management

- Establish a link between configuration management and risk management by identifying configuration items that are critical to mitigating specific risks.
- Ensure that changes in software configurations are assessed for their impact on identified risks.

4. Tools and Techniques

- Consider using version control systems (e.g., Git, Subversion) for SCM and risk management software (e.g., RiskWatch, RiskyProject) for tracking and managing risks.
- Utilize project management tools (e.g., JIRA, Trello) to integrate SCM and risk management activities into the overall project workflow.

EXPERIMENT 9

Objective : Study and use any Design phase CASE tool. Performing the Design by using any Design phase CASE tools

Study and Usage of Any Design Phase CASE Tool

1. Overview of CASE Tools

- **Definition:** Computer-Aided Software Engineering (CASE) tools assist in software development by providing automated support for software design, coding, testing, and maintenance.
- **Types of CASE Tools:**
 - **Upper CASE Tools:** Focus on the early stages of software development, such as requirements gathering and design.
 - **Lower CASE Tools:** Support coding, testing, and maintenance.
 - **Integrated CASE Tools:** Provide support throughout the entire software development lifecycle.

2. Key Features of Design Phase CASE Tools

- **Diagramming Capabilities:** Support for UML diagrams (e.g., use case diagrams, class diagrams, sequence diagrams).
- **Code Generation:** Ability to generate code from design specifications.
- **Documentation Generation:** Automated generation of design documentation.
- **Version Control:** Management of design changes and revisions.
- **Collaboration Features:** Support for teamwork and communication among developers.

3. Popular Design Phase CASE Tools

- **StarUML:** A UML modeling tool that supports various diagram types and allows for code generation.
- **Visual Paradigm:** A comprehensive tool that supports modeling, code generation, and project management.
- **Enterprise Architect:** Offers extensive support for UML and other modeling standards, along with project management features.
- **Lucidchart:** A web-based diagramming tool that facilitates collaborative design and sharing.

Performing the Design Using Any Design Phase CASE Tool

1. Selecting a CASE Tool

- Choose a CASE tool based on your project requirements, team preferences, and budget.
- Consider factors such as ease of use, functionality, and compatibility with other tools.

2. Design Process Using a CASE Tool

- **Step 1: Define Requirements**
 - Gather and document functional and non-functional requirements for the software project.
- **Step 2: Create Use Case Diagrams**
 - Use the CASE tool to create use case diagrams that capture user interactions with the system.
- **Step 3: Develop Class Diagrams**
 - Design the class structure by creating class diagrams that represent the system's entities and relationships.
- **Step 4: Sequence Diagrams**
 - Model the interactions between objects over time using sequence diagrams, illustrating how the system behaves in various scenarios.
- **Step 5: Generate Documentation**
 - Utilize the tool's documentation features to generate design documents that outline the architecture, diagrams, and specifications.
- **Step 6: Review and Iterate**
 - Conduct design reviews with stakeholders to gather feedback and make necessary adjustments.
- **Step 7: Code Generation (if applicable)**
 - Use the code generation feature of the CASE tool to produce initial code from the design models.

3. Collaboration and Version Control

- Encourage team collaboration through shared workspaces in the CASE tool.
- Utilize version control features to manage changes and track the evolution of the design.

EXPERIMENT 10

Objective: Develop test cases for unit testing and integration testing

Unit Testing Test Cases

Test Case ID	Test Case Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_U1	Test Add Cash Function	Verify that the add cash function correctly adds cash.	The cash counter system is initialized.	1. Call <code>addCash(50.00)</code> . 2. Observe the new cash balance.	New balance = 50.00	(To be filled)	(To be filled)
TC_U2	Test Remove Cash Function	Verify that the remove cash	The cash counter has a balance of 100.00.	1. Call <code>removeCash(30.00)</code> . 2. Observe the new cash balance.	New balance = 70.00	(To be filled)	(To be filled)

Test Case ID	Test Case Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
		function deducts cash correctly.					
TC_U3	Test Check Cash Balance Function	Verify that the function correctly returns the current cash balance.	The cash counter has a balance of 70.00.	1. Call <code>checkBalance()</code> . 2. Observe the returned cash balance.	Returned balance = 70.00	(To be filled)	(To be filled)

Integration Testing Test Cases

Test Case ID	Test Case Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_I1	Test Add Cash and Check Balance Integration	Verify that adding cash updates the cash balance correctly.	The cash counter is initialized with a balance of 0.00.	1. Call <code>addCash(100.00)</code> . 2. Call <code>checkBalance()</code> . 3. Observe the returned balance.	Returned balance = 100.00	(To be filled)	(To be filled)
TC_I2	Test Remove Cash and Check Balance Integration	Verify that removing cash updates the cash balance correctly.	The cash counter has a balance of 200.00.	1. Call <code>removeCash(50.00)</code> . 2. Call <code>checkBalance()</code> . 3. Observe the returned balance.	Returned balance = 150.00	(To be filled)	(To be filled)
TC_I3	Test Concurrent Cash Addition and Removal	Verify that the cash counter correctly handles simultaneous operations.	The cash counter has a balance of 100.00.	1. Start two threads: one calling <code>addCash(50.00)</code> and the other calling <code>removeCash(30.00)</code> . 2. Call <code>checkBalance()</code> .	Returned balance = 120.00	(To be filled)	(To be filled)

EXPERIMENT 11

Objective :Develop test cases for various white box and black box testing techniques.

Testing Type	Test Case Description	Expected Output	Actual Output	Pass/Fail
White Box Testing	Verify if the account balance updates after a deposit is made	The account balance should update correctly	The account balance updated correctly	Pass
White Box Testing	Check if user credentials are correctly authenticated	The system should authenticate valid users	The system authenticated valid users	Pass
Black Box Testing	Validate that the user can successfully transfer funds between accounts	Funds should be transferred between accounts without error	Funds transferred successfully	Pass
Black Box Testing	Ensure that invalid password input prompts an error message	An error message should be displayed for invalid input	Error message displayed	Pass
White Box Testing	Check if monthly statements are generated correctly for a user account	Monthly statement should include all transactions for the month	Monthly statement generated correctly	Pass
Black Box Testing	Verify if the system displays correct account balance after a withdrawal	The correct account balance should display after a withdrawal	Correct account balance displayed	Pass

EXPERIMENT 12

Objective : Areas for future development

Future Enhancements:

The system can be expanded to include additional features such as:

1. Mobile banking functionality.
2. Multi-factor authentication for enhanced security.
3. Support for international transactions.
